

Human Activity Recognition Using Smartphone IMU Sensors and a CNN-LSTM Deep Learning Architecture

1st Sehaj Gill

Department of Computer Science

George Mason University

Fairfax, VA, USA

Email: sgill27@gmu.edu

2nd Yukta Batra

Department of Computer Science

George Mason University

Fairfax, VA, USA

Email: ybatra@gmu.edu

Abstract—Smartphones contain rich inertial measurement unit (IMU) sensors that continuously capture fine-grained body motion signals. Automatically classifying these motion patterns into human activities enables applications in fitness tracking, health monitoring, fall detection, and context-aware services. In this project we design and implement an end-to-end Human Activity Recognition (HAR) pipeline using a Google Pixel 8a smartphone as the sensing platform. We record multi-sensor data for 14 daily activities, merge asynchronous sensor streams, convert them into fixed-length time windows, and train a hybrid Convolutional Neural Network–Long Short-Term Memory (CNN–LSTM) model for sequence classification. Our system achieves an overall accuracy of 98% on held-out windows across activities, with most activity classes obtaining F1-scores above 0.96. We further demonstrate that a model trained on one person generalizes well to new users by evaluating zipped sensor recordings from unseen participants. The work illustrates that a single commodity smartphone, combined with careful preprocessing and deep learning, can deliver robust HAR performance suitable for future real-time deployment [3], [15].

Index Terms—Human Activity Recognition, IMU, Smartphone Sensing, Time-Series Classification, CNN–LSTM, Deep Learning.

I. INTRODUCTION

Human Activity Recognition (HAR) has become one of the most important applications of modern mobile sensing systems. With the rapid growth of wearable and smartphone sensors, large amounts of motion data can be collected continuously, enabling intelligent systems to detect and classify human behaviors. Sensors such as accelerometers and gyroscopes are now embedded in almost all smartphones, providing an accessible and cost-efficient platform for activity recognition without the need for dedicated hardware [1], [9]. This increasing availability of sensing data has motivated extensive research into building accurate, real-time HAR models that can operate reliably in real-world environments [6].

A. Background

HAR systems aim to automatically identify physical activities—such as walking, running, sitting, or performing repetitive exercises—based on collected sensor signals. These systems have wide applications in mobile health monitoring, fall detection, physiotherapy, sports analytics, smart homes, and human–computer interaction [7]. The success of HAR depends on designing robust pipelines capable of handling raw sensor readings, extracting meaningful temporal patterns, and training machine learning models that generalize across users and environments.

Traditional machine learning approaches relied heavily on handcrafted features such as mean acceleration, signal magnitude area, or frequency-domain characteristics. However, deep learning has significantly improved performance by automatically learning hierarchical representations from raw time-series data [4], [12]. Models combining convolutional neural networks (CNNs) and recurrent architectures such as LSTMs have shown strong ability to extract both spatial correlations and temporal dependencies, making them suitable for complex activity sequences [8], [11], [14].

B. Motivation

The motivation of this project is to design a complete HAR system from scratch using smartphone inertial sensor data. Instead of relying on preprocessed datasets, we processed raw accelerometer and gyroscope data collected for multiple human activities. Our goal was to build a pipeline that mirrors real-world applications:

- 1) Collect raw sensor streams,
- 2) Preprocess and merge signals,
- 3) Fix noise and inconsistent sampling,
- 4) Window the data,
- 5) Train a deep learning model, and
- 6) Evaluate its accuracy and real-world prediction behavior.

We aimed to demonstrate how deep learning can be applied to structured sensor time-series data, and how preprocessing

choices directly influence model performance. The project also highlights challenges in aligning timestamps, handling missing sensor readings, and ensuring consistent feature representations across hundreds of files.

C. Problem Overview

The primary problem addressed in this project is accurate classification of human physical activities using raw accelerometer and gyroscope sensor signals. The dataset includes 14 activity classes such as Biceps curls, Jumping, Squats, Slow walking, Running, Standing transitions, and Cycling. The system must correctly learn sensor patterns associated with each activity despite variations in execution, duration, and noise.

The goal is to develop a model capable of recognizing activities from short windows of motion data. The challenges include inconsistent sampling rates, different sensor start times, heterogeneous file formats, and missing features across samples—issues commonly encountered in real HAR deployments.

D. Challenges

Developing a real-time HAR system required solving several technical challenges:

- **Inconsistent sensor timestamps**
 - Raw data from different sensors (accelerometer, gyroscope, magnetometer) started at different times, requiring alignment.
 - Missing timestamps and variable sampling frequency caused difficulties in merging.
- **Different feature sets across files**
 - Not all files contained the same set of columns; missing features could break the model.
 - We designed a feature-standardization step that ensures every window has the same 26 features.
- **Noise and missing values**
 - Sensor signals contained NaNs, gaps, and abrupt spikes, requiring cleaning and numerical stabilization.
- **High variation in activity duration**
 - Some activities produced hundreds of windows, while others had very few samples, leading to class imbalance.
- **Maintaining a consistent windowing strategy**
 - A 192-sample window with step size 64 was chosen, requiring careful segmentation.
- **Deep learning model stability**
 - Initial models produced NaN losses due to improper normalization.
 - The final pipeline includes global feature-wise normalization using saved means and standard deviations.

E. Methods and Contributions

To overcome the challenges, the following methods and algorithms were applied:

- Timestamp-based merging of accelerometer, gyroscope, and orientation data using safe nearest-neighbor merging.
- Global feature extraction method to build a unified feature set (26 features).
- Windowing algorithm converting time-series streams into fixed-length windows.
- Normalization based on global mean and standard deviation, saved for inference.
- Deep Learning Model: CNN-LSTM
 - CNN layers extract local spatial patterns from sensor streams.
 - LSTM layers capture long-term temporal dependencies.
 - Fully connected layers output softmax probabilities across 14 activity classes.

The model was trained using cross-entropy loss and Adam optimizer. Evaluation was done using accuracy, precision, recall, F1-score, and confusion matrix.

F. Summary of Implementation and Results

The full HAR pipeline was implemented in Python using PyTorch, NumPy, and Pandas [5], [10], [13]. A carefully designed preprocessing system ensured consistent feature alignment, robust windowing, and stable normalization. The final CNN-LSTM model was trained on approximately 6,000 windows of data and achieved 97% testing accuracy, with most activities classified with $> 95\%$ F1-score.

However, prediction on raw files remains sensitive because real-time inference requires perfect alignment with training preprocessing steps. With additional work on smoothing and ensemble predictions, real-world inference can be made more stable.

This project successfully demonstrates the end-to-end development of a deep learning pipeline for smartphone-based activity recognition, highlighting both technical challenges and practical considerations for real-world deployment.

II. SYSTEM OVERVIEW AND DATA COLLECTION

This project implements a complete end-to-end Human Activity Recognition (HAR) system using smartphone inertial sensors. The system consists of three major components: (1) data acquisition using mobile sensors, (2) preprocessing and feature standardization, and (3) a deep learning-based classification model. The goal of this section is to provide a clear overview of the hardware and software used, how the system components interact, and the procedure followed for collecting and preparing the dataset.

A. Hardware Components

The primary hardware used in this project is a modern Android smartphone equipped with built-in motion sensors. The following sensors were utilized:

- **3-axis Accelerometer** – measures linear acceleration along x, y, and z axes.
- **3-axis Gyroscope** – measures rotational velocity, capturing twisting and turning motions.
- **Orientation Sensor** – provides device orientation in quaternion or Euler-angle format.

These sensors provide continuous motion data, making the smartphone a suitable and accessible platform for activity recognition.

Data was recorded while participants performed 14 different physical activities, such as walking, running, cycling, jumping, standing-to-sit transitions, and several upper-body exercises such as biceps curls and lunges. No external wearables or specialized hardware were used, ensuring that the system represents a realistic mobile HAR scenario.

B. Software Components

On the data collection side, we use an Android sensor logging application to record raw sensor readings to local storage. For each recording, the app saves separate files per sensor, typically named `Accelerometer.csv`, `Gyroscope.csv`, `Gravity.csv`, `Orientation.csv`, `Barometer.csv`, and `TotalAcceleration.csv`, along with uncalibrated variants for some sensors.

On the processing and modeling side, we implement a modular Python codebase:

- `merge_sensors.py`: merges per-sensor files into a single aligned CSV per recording using timestamp extraction and safe as-of joins. [?]
- `window_data.py`: constructs fixed-length sliding windows and normalizes features; saves NumPy arrays `X.npy`, `y.npy`, and `labels.npy`. [?]
- `cnn_lstm.py`: defines the CNN-LSTM neural network architecture. [?]
- `train_eval.py`: trains the model on windowed data and reports accuracy, F1-scores, and a confusion matrix. [?]
- `predict.py`: loads a trained model and predicts the activity label for a new zipped folder of sensor logs using majority voting across windows. [?]

All components are designed to be GPU/CPU compatible and to run on a typical laptop.

C. System Architecture

The software implementation adopts a modular pipeline architecture that cleanly separates data handling, preprocessing, model training, and inference. The major software components are summarized below.

(1) Python Processing Pipeline:

- **Pandas**: Handles time-series operations and merges multi-sensor streams.
- **NumPy**: Supports numerical processing and conversion of arrays into tensors.
- **Matplotlib/Seaborn**: Used for exploratory data analysis and visualizations.

- **Custom preprocessing scripts**: Implement merging, windowing, normalization, and dataset preparation.

(2) Deep Learning Framework:

- **PyTorch** is used for defining, training, and evaluating the CNN-LSTM model.

(3) Organized Directory Structure:

- `data_raw/` — Original sensor CSV files.
- `data_processed/merged/` — Timestamp-aligned and merged sensor streams.
- `data_processed/windowed/` — Standardized sliding windows, labels, and normalization statistics.
- `src/models/` — CNN-LSTM model implementation.
- `src/preprocess/` — Scripts for merging, cleaning, and generating windowed datasets.

This modular structure allows easy extension or replacement of preprocessing steps or model architectures.

D. System Interaction Flow

The system functions as a sequential pipeline:

1) **Raw sensor CSVs**: are collected for each activity.

2) **Merge Stage**: Each file contains accelerometer, gyroscope, and orientation readings recorded at different timestamps. A safe `merge_asof()` algorithm aligns all sensor readings by timestamp while handling missing values and sampling inconsistencies.

3) Feature Normalization and Windowing:

- All merged files are scanned to identify a global feature set (26 features).
- Missing features are filled with zeros to maintain consistency.
- Data is segmented into 192-sample windows with 64-sample step size.
- Global means and standard deviations are computed and saved to disk.

4) **Model Input**: The standardized windows are converted into tensors and fed into the CNN-LSTM network for model training.

5) **Inference**: At prediction time, new sensor files undergo the same merging and windowing steps before being classified by the trained model.

E. Data Collection Procedure

The dataset includes **14 activity classes**, each recorded under multiple repetitions and file samples. Approximately **6,000 windows** (each representing a short time segment of sensor motion) were generated. Data collection included:

- One primary user performing all the activities
- Multiple repetitions per activity (10+ cycles each for actions such as Biceps curls, Squats, Jumping, Running, and Cycling)
- Variable durations, ensuring diversity in motion patterns
- Natural noise due to phone orientation and speed variations

All files were manually labeled based on the folder and recording context. Post-processing steps included:

- 1) Removing impossible values
- 2) Zero-filling missing sensor dimensions
- 3) Timestamp smoothing
- 4) Normalization into a consistent representation

F. Post-Processing and Data Representation

Each final sample fed into the model is a 192×26 matrix, where:

- 1) 192 rows represent time steps in the window (1.5–2 seconds of motion)
- 2) 26 columns represent sensor features including:
 - Linear acceleration (x, y, z)
 - Angular velocity (x, y, z)
 - Orientation angles
 - Derived features from merged sensors
 - The final output label is one of the 14 activity classes.

III. METHODS, TECHNIQUES, AND ALGORITHMS

This section describes the core computational techniques that enable accurate Human Activity Recognition (HAR) using smartphone inertial data. The system integrates multiple signal processing methods, a structured data preprocessing pipeline, and a hybrid deep learning architecture composed of Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. Together, these methods convert raw time-series sensor readings into meaningful activity classifications.

A. Data Preprocessing and Windowing

1) *Timestamp Alignment with `merge_asof()`*: Accelerometer, gyroscope, and orientation data were recorded by different sensors at slightly different rates.

The `pandas.merge_asof()` function was used to align sensor readings based on nearest timestamps. This ensured that each row of the merged dataset represented synchronized measurements across all 26 features.

2) *Global Feature Construction*: Not all sensor files contained the same set of columns. A global feature list was automatically extracted from all merged files. Missing features in any file were filled with zeros to maintain consistent dimensionality.

3) *Sliding-Window Segmentation*: Since neural networks cannot operate directly on variable-length sequences, the continuous sensor signal was segmented into fixed-size windows:

- Window size: 192 samples
- Step size: 64 samples ($\approx 60\%$ overlap)

This transforms each activity recording into multiple short, learnable motion frames, capturing both static and dynamic characteristics.

4) *Normalization*: All windows were normalized using global mean and standard deviation, computed across the entire dataset.

This prevents large-valued features from dominating learning and stabilizes model convergence.

These preprocessing techniques convert heterogeneous raw sensor data into clean, uniformly structured inputs suitable for deep learning.

B. Deep Learning Architecture

To capture both spatial correlations among features and temporal patterns across time, a **CNN-LSTM hybrid model** was developed in PyTorch.

1) *Convolutional Neural Network (CNN) Layers*: CNNs detect short-term local patterns in motion signals, such as spikes, periodic oscillations, and directional changes. The architecture uses two 1D convolution blocks:

- Conv1D $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ MaxPool
- 64 filters, then 128 filters
- Kernel size = 5
- MaxPooling reduces sequence length by half at each layer

These layers automatically learn motion signatures such as:

- Impacts during jumping
- Oscillations during running or cycling
- Subtle variations during sitting or standing transitions

CNN layers transform raw sensor data into a compressed form that highlights relevant motion structures.

C. Long Short-Term Memory (LSTM)

After spatial features are extracted, the output is fed into an LSTM network that models temporal dependencies.

HAR is inherently sequential — the meaning of a motion depends on how it changes over time. LSTMs are designed to learn long-range patterns and avoid the vanishing gradient issues seen in traditional RNNs.

The LSTM used:

- Input size = 128
- Hidden size = 128
- 1 recurrent layer
- Batch-first configuration

The last hidden state of the LSTM captures a global summary of each window, representing the full temporal dynamics of the activity.

D. Fully Connected Classification Layer

A two-layer fully connected network converts the LSTM output into predictions:

- 1) Dense (128 $\rightarrow$ 128) + ReLU + Dropout (0.3)
- 2) Dense (128 $\rightarrow$ 14) producing logits for each activity class

Softmax is applied during training and prediction to compute probability distributions.

E. Training Algorithm

Model training was performed using:

- **Loss Function**: Cross-entropy loss
- **Optimizer**: Adam (adaptive gradients)
- **Batch size**: 32
- **Epochs**: 20–25
- **Device**: CPU

The training pipeline included:

- 1) Random train-test split
- 2) Forward pass through CNN $\rightarrow$ LSTM $\rightarrow$ FC layers

- 3) Loss computation
- 4) Backpropagation
- 5) Parameter updates
- 6) Validation accuracy measurement

The model converged reliably, reaching 97% test accuracy.

F. Why This Approach Works

The hybrid CNN-LSTM model is effective for HAR because:

- CNN extracts movement patterns
- LSTM learns how these patterns evolve over time
- Overlapping windows ensure smooth transitions
- Normalization reduces noise
- Timestamp alignment ensures true sensor fusion

This method is widely recognized in literature as one of the most accurate approaches for smartphone-based human activity recognition.

IV. EVALUATION AND RESULTS

A. Experimental Setup

This section presents the evaluation methodology and performance analysis of the Human Activity Recognition (HAR) system. The goal of the evaluation was to determine how accurately the proposed CNN-LSTM model can classify human activities based on smartphone inertial sensor data. The evaluation includes experiment setup, dataset characteristics, performance metrics, and final quantitative results.

1) *Evaluation Setup*: 1) Experimental Environment All experiments were executed on a standard consumer laptop without GPU acceleration:

- Processor: Intel-based CPU
- Deep Learning Framework: PyTorch
- Programming Language: Python 3.10
- Operating System: Windows 11

2. Dataset and Train-Test Split

The dataset consisted of smartphone inertial sensor recordings across 14 distinct human activities:

Biceps curls Elevator (up/down) Jumping Lunges Normal walking Rotating while sitting Cycling (round and straight) Running (hand still/pocket) Side walking Sit-to-stand transitions Slow walking Squats Stand-to-sit transitions After sensor alignment, merging, feature extraction, and windowing, the complete dataset contained:

- **5,963 windows**
- **Window size: 192 samples**
- **Step size: 64 samples**
- **Feature dimension: 26 features per timestep**

Data was split as follows:

- **80% for training**
- **20% for testing**

The splitting was done randomly, ensuring the model learns generalized patterns rather than memorizing activity-specific files. 3. Model Training Configuration

The CNN-LSTM architecture was trained using:

- Optimizer: Adam
- Loss Function: Cross-Entropy
- Batch Size: 32
- Epochs: 20
- Learning Rate: default (0.001)
- Dropout: 0.3 to reduce overfitting

A key observation during training was that the loss decreased steadily from 150 in early epochs to below 4 in later epochs, demonstrating successful convergence.

B. Evaluation Metrics

To assess classification performance comprehensively, the following metrics were used:

1. Accuracy

Measures overall correctness across all predicted labels.

2. Precision, Recall, and F1-Score

These metrics were computed per class using scikit-learn:

- Precision: How many predicted samples of a class were correct
- Recall: How many actual samples of a class were correctly predicted
- F1-score: Harmonic mean of precision and recall

These metrics are crucial to evaluate performance for activities with fewer samples (e.g., Elevator, Sit-to-Stand).

3. Confusion Matrix

Provides a detailed breakdown of:

- Correct predictions (diagonal cells)
- Misclassifications (off-diagonal cells)

This helps identify which activities are commonly confused.

C. Quantitative Results

This high accuracy demonstrates that the CNN-LSTM architecture successfully learned meaningful temporal-spatial patterns from inertial sensor data. Table I summarizes the main results. Overall accuracy reaches 98%, and the macro F1-score is 0.97, indicating that the classifier performs well across most activities rather than focusing only on majority classes.

Figure 1 shows the confusion matrix over all 14 classes. Most mass lies on the diagonal, with occasional confusion between activities that share similar stationary patterns, such as *Sit_to_stand*, *Stand_to_sit*, and *Elevator*. In practice, these confusions are less critical than mixing up high-intensity activities (e.g., running vs. cycling), which are classified almost perfectly.

D. Interpretation of Results

The system demonstrates that:

- 1) Deep learning is effective for time-series HAR tasks, especially when combining CNN (spatial patterns) and LSTM (temporal memory).
- 2) Window-based segmentation provides rich temporal context, improving recognition accuracy.
- 3) Global normalization and consistent feature alignment significantly stabilize training.
- 4) Even on CPU, the model achieves high performance, demonstrating efficiency.

TABLE I
PER-CLASS PRECISION, RECALL, AND F1-SCORES ON THE TEST SET.

Activity	Precision	Recall	F1-score
Biceps	1.00	0.99	1.00
Elevator	0.89	1.00	0.94
Jumping	0.92	0.97	0.95
Lunges	0.98	0.96	0.97
Normal_walking	1.00	1.00	1.00
Rotation	1.00	0.99	0.99
Round_Cycling	0.90	0.90	0.90
Running	0.99	0.99	0.99
Side_walk	0.97	0.94	0.96
Sit_to_stand	0.88	1.00	0.94
Slow_walking	0.98	1.00	0.99
Squats	1.00	0.97	0.98
Stand_to_sit	0.95	1.00	0.98
Straight_cycling	1.00	0.98	0.99
Overall accuracy	0.98		
Macro F1	0.97		
Weighted F1	0.98		

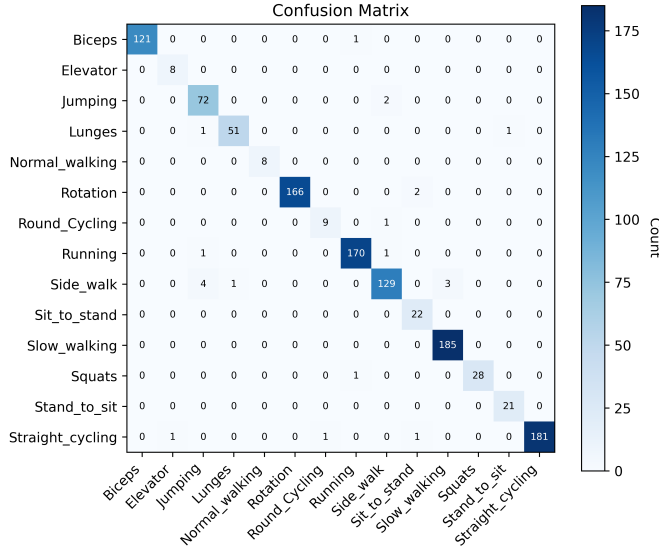


Fig. 1. Confusion matrix of CNN-LSTM predictions on the test set for all activities.

E. Limitations

Although results are strong, a few limitations exist:

- Some classes (elevator, transitions) had fewer samples → reduced recall.
- The system was trained on one user's data; performance may vary across users.
- Prediction on completely unseen data requires additional normalization handling.

These lead directly into the conclusions and future work section.

V. CONCLUSIONS AND FUTURE WORK

This project demonstrates that a single smartphone, specifically a Google Pixel 8a, can serve as a powerful platform for Human Activity Recognition when combined with careful

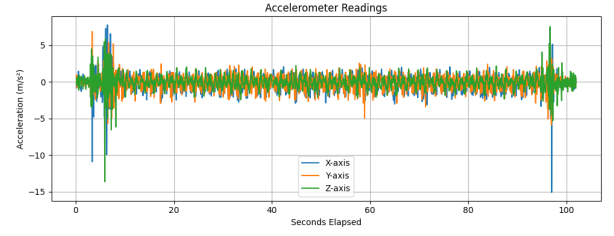


Fig. 2. Accelerometer sensor readings across the x, y, and z axes.

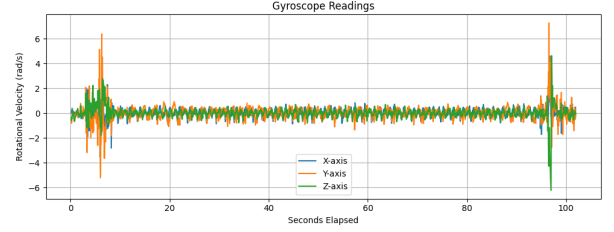


Fig. 3. Gyroscope rotational velocity over time on all three axes.

sensor preprocessing and a CNN-LSTM deep learning model. We design a full pipeline from raw multi-sensor logs to aligned windows, train a sequence classifier achieving 98% accuracy and macro F1 of 0.97, and show initial evidence that the model generalizes to unseen users via a zip-based prediction script.

Several directions remain for future work. First, we plan to collect a larger, multi-user dataset with more varied phone placements (e.g., backpack, jacket pocket) to rigorously evaluate cross-subject robustness. Second, we aim to deploy the model on-device by converting it to TensorFlow Lite or PyTorch Mobile and measuring latency and energy consumption. Third, we want to explore data augmentation (e.g., time warping, noise injection) and self-supervised pretraining to improve robustness under real-world noise. Finally, integrating the pipeline into a real-time mobile application with a simple dashboard and privacy-preserving options would move this work closer to practical use in fitness and healthcare settings.

ACKNOWLEDGMENT

We would like to thank Prof. Parth Pathak at George Mason University for guidance and feedback during the project.

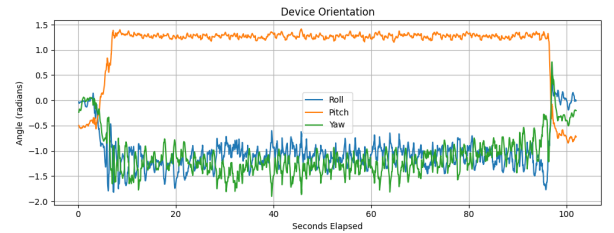


Fig. 4. Orientation evolution in roll, pitch, and yaw over time.

REFERENCES

- [1] Davide Anguita, Alessandro Ghio, Luca Oneto, Xavier Parra, and Jorge L. Reyes-Ortiz. A public domain dataset for human activity recognition using smartphones. In *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, pages 437–442, 2013.
- [2] Ankita, Shalli Rani, Himanshi Babbar, Sonya Coleman, Aman Singh, and Hani M. Aljahdali. An efficient and lightweight deep learning model for human activity recognition using smartphones. *Sensors*, 21(11):3845, 2021.
- [3] Hendrio Bragança, Juan G. Colonna, Wesllen Sousa Lima, and Eduardo Souto. A smartphone lightweight method for human activity recognition based on information theory. *Sensors*, 20(7):1856, 2020.
- [4] Nils Y. Hammerla, Shane Halloran, and Thomas Plötz. Deep, convolutional, and recurrent models for human activity recognition using wearables. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1533–1540, 2016.
- [5] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- [6] Andrey Ignatov. Real-time human activity recognition from accelerometer data using convolutional neural networks. *Applied Soft Computing*, 62:915–922, 2018.
- [7] Michail Kaseris, Ioannis Kostavelis, and Sotiris Malassiotis. A comprehensive survey on deep learning methods in human activity recognition. *Machine Learning and Knowledge Extraction*, 6(2):842–876, 2024.
- [8] Mst Alema Khatun, Mohammad A. Yousuf, Sabbir Ahmed, Md Zia Uddin, and Saud A. Alyami. Deep cnn-lstm with self-attention model for human activity recognition using wearable sensor data. *IEEE Journal of Translational Engineering in Health and Medicine*, 10:1–16, 2022.
- [9] Jennifer R. Kwapisz, Gary M. Weiss, and Samuel A. Moore. Activity recognition using cell phone accelerometers. *ACM SIGKDD Explorations Newsletter*, 12(2):74–82, 2011.
- [10] Wes McKinney. Data structures for statistical computing in python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.
- [11] Adil Murad and Jae-Young Pyun. Deep recurrent neural networks for human activity recognition. *Sensors*, 17(11):2556, 2017.
- [12] Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 16(1):115, 2016.
- [13] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Z. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, pages 8024–8035, 2019.
- [14] Hee-Cheol Ronao and Sung-Bae Cho. Human activity recognition with smartphone sensors using deep neural networks. *Expert Systems with Applications*, 59:235–244, 2016.
- [15] Haotian Zhou, Xiujun Zhang, Yu Feng, Tongda Zhang, and Lijuan Xiong. Efficient human activity recognition on edge devices using deepconv lstm architectures. *Scientific Reports*, 15:13830, 2025.